

## مونتاژ سخت‌افزاری

### مرحله ۱: قرار دادن برد ESP32 روی بردبرد

برد ESP32 به صورت عمودی و در مرکز بردبرد قرار داده شد به گونه‌ای که دو ردیف پایه‌ها در دو سمت شیار میانی بردبرد قرار بگیرند. این کار برای جلوگیری از اتصال ناخواسته پایه‌ها و سهولت سیم‌کشی انجام شد.

### مرحله ۲: ایجاد ریل تغذیه مشترک

یکی از پایه‌های GND برد ESP32 به ریل منفی بردبرد متصل شد. پایه 3.3V برد ESP32 نیز به ریل مثبت بردبرد متصل گردید. از این دو ریل برای تغذیه تمام سنسورها و ماژول‌ها استفاده شد.

### مرحله ۳: اتصال سنسور DHT11

سنسور DHT11 روی بردبرد قرار گرفت به گونه‌ای که پایه‌ها در ستون‌های مجزا باشند. اتصالات به این صورت انجام شد:

پایه VCC سنسور به ریل 3.3V

پایه GND سنسور به ریل GND

پایه DATA سنسور به GPIO27 (P27) برد ESP32

به منظور پایداری خط داده، یک مقاومت  $10k\Omega$  بین پایه DATA و ریل 3.3V قرار داده شد (Pull-up).

### مرحله ۴: اتصال نمایشگر TFT مبتنی بر SPI

نمایشگر TFT روی بردبرد قرار گرفت و پایه‌های آن به صورت زیر متصل شدند:

GND → GND

VCC → 3.3V

SCK → GPIO18 (P18)

SDA/MOSI → GPIO23 (P23)

DC/A0 → GPIO21 (P21)

RST → GPIO4 (P4)

CS → GPIO5 (P5)

LED → 3.3V

ابتدا تنها سیم‌های سیگنال متصل شدند و اتصال VCC و LED در مرحله پایانی انجام گرفت.

#### مرحله ۵: اتصال میکروفون دیجیتال INMP441 (I2S)

میکروفون INMP441 روی بردبرد قرار گرفت و پایه‌های آن به صورت زیر متصل شدند:

GND → GND

SD → GPIO32 (P32)

WS → GPIO25 (P25)

SCK → GPIO26 (P26)

L/R → GND (انتخاب کانال Left)

VDD → 3.3V (در مرحله نهایی)

اتصالات سیگنال ابتدا انجام شد و اتصال تغذیه در مرحله آخر صورت گرفت.

#### مرحله ۶: بررسی نهایی اتصالات قبل از اعمال تغذیه

پیش از اتصال منبع تغذیه، تمامی اتصالات از نظر صحت پایه‌ها، عدم استفاده از پایه‌های Boot، و مشترک بودن زمین‌ها بررسی شدند. همچنین اطمینان حاصل شد که هیچ پایه‌ای به 5V متصل نشده است.

## مرحله ۷: اتصال تغذیه از طریق USB

پس از اطمینان از صحت کامل سیم‌کشی، برد ESP32 از طریق پورت USB به منبع تغذیه متصل شد. در این مرحله، تغذیه ماژول‌ها فعال شد و مدار آماده تست نرم‌افزاری گردید.

## برنامه نویسی و تست برد

### مرحله 1: برنامه‌نویسی LCD

کتابخانه‌های Adafruit\_GFX و Adafruit\_ST7735 اضافه و پایه‌های ارتباطی LCD مطابق سیم‌کشی تعریف شد.

راه‌اندازی اولیه نمایشگر انجام و توابع لازم برای نمایش متن/عدد پیاده‌سازی شد.

### مرحله 2: تست LCD

صفحه با fillScreen پاک‌سازی و چند متن/عدد ثابت در مختصات مشخص نمایش داده شد. با بررسی رنگ، جهت نمایش و خروجی نهایی، صحت اتصال و عملکرد LCD تأیید شد.

### مرحله 3: برنامه‌نویسی DHT11

کتابخانه DHT.h اضافه و DHTPIN و DHTTYPE تنظیم و dht.begin() اجرا شد. خواندن دما و رطوبت انجام و داده‌ها برای نمایش یا ارسال آماده‌سازی شد.

### مرحله 4: تست DHT11

مقادیر دما و رطوبت روی سریال و LCD بررسی شد تا منطقی بودن داده‌ها مشخص شود. در صورت خطای خواندن (مثل NaN) پیام خطا برای تشخیص سریع مشکل نمایش داده شد.

### مرحله 5: برنامه‌نویسی شبکه

اتصال WiFi برقرار و وضعیت اتصال و IP گزارش شد.  
وبسوکتهای کنترل و صوت ایجاد و رویدادهای اتصال/قطع و پیامهای کنترلی مدیریت شد.

### مرحله 6: برنامه نویسی میکروفون

ورودی I2S با نرخ نمونه برداری مناسب تنظیم و دریافت دادهای صوتی فعال شد.  
دادهها به PCM16 تبدیل و برای ارسال پیوسته به صورت بستههای صوتی آماده شد.

### مرحله 7: تست میکروفون

پایداری خواندن I2S و مقدار bytesRead بررسی شد تا افت داده یا قطع شدن رخ ندهد.  
ارسال صوت روی وبسوکت تست و فرمانهای START/STOP برای کنترل ضبط بررسی شد.

### مرحله 8: سرهم کردن بخشها و تست نهایی

بخشهای LCD، DHT11، شبکه و میکروفون در یک ساختار واحد ادغام شد.  
تست نهایی شامل نمایش پایدار دما/رطوبت، اتصال شبکه، دریافت فرمانها و استریم صوت بدون وقفه انجام شد.

## کدنویسی پنل سرور

### مرحله 1: طراحی ساختار پنل و مسیرها

صفحههای اصلی پنل (Dashboard، لیست ضبطها، تنظیمات) مشخص و مسیرهای URL برای هر بخش تعریف شد.  
ساختار پوشهها برای قالبها (templates)، فایلهای استاتیک (static) و خروجیها (recordings/texts) آماده سازی شد.

### مرحله 2: پیاده سازی سرور HTTP پنل

سرور وب با فریمورک (مثل Flask) راه اندازی و رندر صفحات با render\_template انجام شد.  
مسیرهای اصلی مثل صفحه ورود، صفحه پنل و صفحه نمایش ضبطها به برنامه اضافه شد.

### مرحله 3: پیاده سازی احراز هویت پنل

رمز عبور مدیر (Teacher) تعریف و مکانیزم لاگین/لاگ‌اوت با session پیاده‌سازی شد.  
برای مسیرهای حساس، بررسی دسترسی انجام شد تا فقط کاربر مجاز وارد بخش‌های مدیریتی شود.

#### مرحله 4: پیاده‌سازی WebSocket برای ارتباط زنده

سرور WebSocket برای دریافت/ارسال پیام‌های بلادرنگ (وضعیت دستگاه و کنترل‌ها) راه‌اندازی شد.  
رویدادهای اتصال/قطع، دریافت پیام و ارسال فرمان‌های کنترلی مدیریت و لاگ‌گذاری شد.

#### مرحله 5: مدیریت فرمان‌ها و وضعیت دستگاه

پیام‌های کنترلی مانند START/STOP و تنظیمات مربوط به دستگاه تعریف و پردازش شد.  
وضعیت فعلی (اتصال، در حال ضبط بودن، خطاها) نگهداری و به پنل نمایش داده شد.

#### مرحله 6: مدیریت فایل‌های صوتی و ذخیره‌سازی

دریافت داده صوتی/فایل‌ها در مسیر مشخص ذخیره و نام‌گذاری استاندارد برای هر ضبط اعمال شد.  
امکان لیست کردن، دانلود و نمایش اطلاعات هر فایل از طریق پنل فراهم شد.

#### مرحله 7: پردازش صوت و تولید متن

پس از پایان ضبط، مراحل پاک‌سازی/تبدیل (مثل تبدیل به PCM/WAV و کاهش نویز) اجرا شد.  
سپس تبدیل گفتار به متن (Speech-to-Text) انجام و خروجی در پوشه متون ذخیره شد.

#### مرحله 8: ساخت خلاصه و خروجی‌های متنی پنل

روی متن استخراج‌شده، تولید خلاصه و ساخت فایل‌های خروجی (speech/summary) انجام شد.  
در پنل، نمایش متن کامل و خلاصه به صورت صفحه جداگانه یا بخش‌بندی‌شده اضافه شد.

#### مرحله 9: طراحی رابط کاربری پنل

صفحه‌ها با HTML/CSS و اجزای کنترلی (دکمه شروع/توقف، لیست ضبط‌ها، وضعیت اتصال) ساخته شد.  
نمایش زنده وضعیت با WebSocket (بدون رفرش صفحه) برای تجربه مدیریتی بهتر اضافه شد.

#### مرحله 10: تست و عیب‌یابی نهایی پنل

سناریوهای اصلی مثل ورود، شروع/توقف، قطع و وصل شبکه، ذخیره فایل و تولید متن تست شد.  
لاگ‌ها و خطاها بررسی و اصلاح شد تا پنل در اجرای طولانی پایدار و بدون اختلال کار کند.

## پیاده سازی ویس به متن

### مرحله 1: تعریف مسیرها و پوشه‌ها

مسیرهای ورودی/خروجی شامل recordings/clean و recordings/texts/summery و recordings/texts/speech مشخص و تنظیم شد. پوشه‌های موردنیاز با `mkdir(..., exist_ok=True)` ساخته شد تا اجرای برنامه بدون خطای نبود مسیر انجام شود.

### مرحله 2: آماده‌سازی کتابخانه‌ها و وابستگی‌ها

کتابخانه‌های پردازش صوت (`librosa`, `soundfile`, `noisereduce`) و تبدیل گفتار به متن (`whisper`) وارد و آماده استفاده شد. برای خلاصه‌سازی متن، ابزار LLM با `langchain_openai` و پیام‌های سیستمی/کاربری تنظیم شد.

### مرحله 3: بارگذاری مدل Whisper

مدل تشخیص گفتار با `whisper.load_model("large-v3")` بارگذاری و برای پردازش چند فایل آماده شد. یک `initial_prompt` برای بهبود دقت در متن فارسی محاوره‌ای تعریف شد.

### مرحله 4: خواندن فایل‌های صوتی و تبدیل به فرمت مناسب

فایل‌های صوتی ورودی خوانده و در صورت نیاز به نرخ نمونه‌برداری هدف (مثلاً 16000) تبدیل شد. صوت به شکل آرایه عددی استاندارد آماده شد تا مراحل پاک‌سازی و تبدیل گفتار اجرا شود.

### مرحله 5: کاهش نویز و پاک‌سازی صوت

برای بهبود کیفیت، کاهش نویز با `noisereduce` روی سیگنال اعمال شد. خروجی پاک‌سازی‌شده در مسیر `recordings/clean` ذخیره شد تا هم قابل بررسی باشد هم ورودی Whisper بهتر شود.

### مرحله 6: تبدیل گفتار به متن

روی فایل پاک‌سازی‌شده، `whisper.transcribe(...)` اجرا و متن کامل گفتار استخراج شد. خروجی متن خام در فایل‌های پوشه `texts/speech` ذخیره شد تا به عنوان متن اصلی قابل استفاده باشد.

## مرحله 7: خلاصه‌سازی متن با مدل زبانی

متن استخراج‌شده به LLM داده شد و با پیام سیستمی مشخص، خلاصه قابل‌استفاده تولید شد. خلاصه در پوشه texts/summery ذخیره شد تا خروجی نهایی پنل/سرور آماده باشد.

## مرحله 8: مدیریت خروجی‌ها و نام‌گذاری فایل‌ها

برای هر ورودی، نام فایل خروجی متن و خلاصه بر اساس نام همان ضبط ساخته شد تا قابل ردیابی باشد.

مسیرها و فایل‌ها طوری مدیریت شد که اجرای چندباره باعث تداخل یا overwrite ناخواسته نشود.

## جمع بندی

به طور کلی و خلاصه در این پروژه ابتدا سخت‌افزار نمایشگر LCD راه‌اندازی شد؛ کتابخانه‌های لازم اضافه و پایه‌های اتصال طبق سیم‌کشی تعریف شدند تا امکان نمایش متن و داده روی صفحه فراهم شود، سپس با چاپ متن و اعداد ثابت و بررسی رنگ و چرخش، عملکرد LCD تست و تأیید شد. بعد از آن سنسور DHT11 پیاده‌سازی شد؛ پایه و نوع سنسور تنظیم و خواندن دما و رطوبت انجام شد تا داده‌ها برای نمایش آماده شوند، سپس با کنترل خروجی روی سریال و LCD و بررسی خطاهایی مثل NaN، صحت عملکرد DHT11 بررسی شد. در مرحله بعد بخش شبکه توسعه داده شد؛ اتصال WiFi برقرار و وضعیت اتصال گزارش شد و ارتباط بلادرنگ از طریق WebSocket برای کنترل دستگاه و انتقال صوت راه‌اندازی شد. همزمان بخش میکروفون با I2S تنظیم شد؛ دریافت پایدار داده صوتی فعال و داده‌ها به PCM16 تبدیل و برای ارسال پیوسته بسته‌بندی شدند، سپس با بررسی پایداری خواندن I2S و تست ارسال روی WebSocket و فرمان‌های START/STOP، عملکرد میکروفون تأیید شد. بعد از آماده شدن اجزای اصلی، همه بخش‌ها در یک ساختار واحد ادغام شدند تا در کنار هم بدون تداخل اجرا شوند و در تست نهایی، نمایش دما/رطوبت روی LCD همراه با اتصال شبکه و استریم صوت پایدار بررسی و نهایی شد. در سمت سرور نیز پنل مدیریتی طراحی و پیاده‌سازی شد؛ صفحات و مسیرها تعریف، سرور HTTP راه‌اندازی و احراز هویت اضافه شد و سپس WebSocket برای کنترل زنده و نمایش وضعیت دستگاه فعال گردید. مدیریت فایل‌های صوتی، ذخیره‌سازی و ارائه لیست/دانلود در پنل انجام شد و پس از هر ضبط، پردازش صوت اجرا شد. در فایل speech.py مسیرها و خروجی‌ها تنظیم شد، مدل Whisper بارگذاری و صوت پس از پاک‌سازی و کاهش نویز به متن تبدیل شد؛ سپس متن برای تولید خلاصه به مدل زبانی ارسال و خروجی‌های متن کامل و خلاصه در پوشه‌های مشخص ذخیره شدند تا در پنل قابل مشاهده و استفاده باشند.